

§ 9.1 词典ADT

9.1.1 操作接口

除通用的接口之外，词典结构主要的操作接口可归纳为表9.1。

表9.1 词典ADT支持的标准操作接口

操 作 接 口	功 能 描 述
get(key)	若词典中存在以key为关键码的词条，则返回该词条的数据对象；否则，返回NULL
put(key, value)	插入词条(key, value)，并报告是否成功
remove(key)	若词典中存在以key为关键码的词条，则删除之并返回true；否则，返回false

实际上，包括Snobol4、MUMPS、SETL、Rexx、Awk、Perl、Ruby、PHP、Java和Python等在内，许多编程语言都以各自不同形式，支持类似于以上词典或映射ADT接口功能的基本数据结构，有的甚至将它们作为基本的数据类型，统称作关联数组（associative array）。

9.1.2 操作实例

比如，可如图9.1所示，将三国名将所对应的词条组织为一个词典结构。其中的每一词条，都由人物的字（style）和姓名（name）构成，分别作为词条的关键码和数据项。



图9.1 三国人物的词典结构

以初始包含关、张、马、黄四将的词典为例，在依次执行一系列操作的过程中，该词典结构内容的变化以及对应的输出如表9.2所示。

表9.2 词典结构操作实例

操 作	词 典 结 构	输 出
size()	(<code>"Yunchang"</code> , <code>"Yu GUAN"</code>) (<code>"Yide"</code> , <code>"Fei ZHANG"</code>) (<code>"Mengqi"</code> , <code>"Chao MA"</code>) (<code>"Hansheng"</code> , <code>"Zhong HUANG"</code>)	4
put(<code>"Bofu"</code> , <code>"Ce SUN"</code>)	(<code>"Yunchang"</code> , <code>"Yu GUAN"</code>) (<code>"Yide"</code> , <code>"Fei ZHANG"</code>) (<code>"Mengqi"</code> , <code>"Chao MA"</code>) (<code>"Hansheng"</code> , <code>"Zhong HUANG"</code>) <u>(<code>"Bofu"</code>, <code>"Ce SUN"</code>)</u>	true
size()	[unchanged]	5
get(<code>"Yide"</code>)	[unchanged]	<code>"Fei ZHANG"</code>
get(<code>"Zilong"</code>)	[unchanged]	NULL

表9.2 词典结构操作实例（续）

操 作	词 典 结 构	输 出
put("Yide", "Fei CHANG")	("Yunchang", "Yu GUAN") ("Yide", "Fei CHANG") ("Mengqi", "Chao MA") ("Hansheng", "Zhong HUANG") ("Bofu", "Ce SUN")	true
size()	[unchanged]	5
get("Yide")	[unchanged]	"Fei CHANG"
remove("Mengqi")	("Yunchang", "Yu GUAN") ("Yide", "Fei CHANG") ("Hansheng", "Zhong HUANG") ("Bofu", "Ce SUN")	"Chao MA"
size()	[unchanged]	4

请特别留意以上第二次put()操作，其拟插入词条的关键码"Yide"，在该词典中已经存在。由该实例可见，插入效果等同于用新词条替换已有词条；相应地，put()操作也必然会成功。这一处理方式被包括Python和Perl在内的众多编程语言普遍采用，但本章采用的约定与此略有不同。跳转表将允许同时保留多个关键码雷同的词条，查找时任意返回其一；散列表则维持原词条不变，返回插入失败标志——也就是说，更接近于映射的规范。

9.1.3 接口定义

这里首先以如代码9.1所示模板类的形式定义词典的操作接口。

```
1 template <typename K, typename V> struct Dictionary { //词典Dictionary模板类
2     virtual int size() const = 0; //当前词条总数
3     virtual bool put(K, V) = 0; //插入词条（禁止雷同词条时可能失败）
4     virtual V* get(K k) = 0; //读取词条
5     virtual bool remove(K k) = 0; //删除词条
6 };
```

代码9.1 词典结构的操作接口规范

其中，所有操作接口均以虚函数形式给出，留待在派生类中予以具体实现。

另外，正如此前所述，尽管词条关键码类型可能支持大小比较，但这并非词典结构的必要条件，Dictionary模板类中的Entry类只需支持判等操作。

9.1.4 实现方法

不难发现，基于此前介绍的任何一种平衡二叉搜索树，都可便捷地实现词典结构。比如，Java语言的java.util.TreeMap类即是基于红黑树实现的词典结构。然而这类实现方式都在不经意中假设“关键码可以比较大小”，故其所实现的并非严格意义上的词典结构。

以下以跳转表和散列表为例介绍词典结构的两种实现方法。尽管它们都在底层引入了某种“序”，但这类“序”只是内部的一种约定；从外部接口来看，依然只有“相等”的概念。

§ 9.2 *跳转表

第2章所介绍的有序向量和第3章所介绍的有序列表，各有所长：前者便于静态查找，但动态维护成本较高；后者便于增量式的动态维护，但只能支持顺序查找。为结合二者的优点，同时弥补其不足，第7章和第8章逐步引入了平衡二叉搜索树，其查找、插入和删除操作均可在 $O(\log n)$ 时间内完成。尽管如此，这些结构的相关算法往往较为复杂，代码实现和调试的难度较大，其正确性、鲁棒性和可维护性也很难保证。

设计并引入跳转表^②（skip list）结构的初衷，正是在于试图找到另外一种简便直观的方式，来完成这一任务。具体地，跳转表是一种高效的词典结构，它的定义与实现完全基于第3章的有序列表结构，其查询和维护操作在平均的意义下均仅需 $O(\log n)$ 时间。

9.2.1 Skiplist模板类

跳转表结构以模板类形式定义的接口，如代码9.2所示。

```
1 #include "../List/List.h" //引入列表
2 #include "../Entry/Entry.h" //引入词条
3 #include "Quadlist.h" //引入Quadlist
4 #include "../Dictionary/Dictionary.h" //引入词典
5
6 template <typename K, typename V> //key、value
7 //符合Dictionary接口的Skiplist模板类（但隐含假设元素之间可比较大小）
8 class Skiplist : public Dictionary<K, V>, public List<Quadlist<Entry<K, V>>*> {
9 protected:
10     bool skipSearch(
11         ListNode<Quadlist<Entry<K, V>>*> &qlist,
12         QuadlistNode<Entry<K, V>>* &p,
13         K& k);
14 public:
15     int size() const { return empty() ? 0 : last()->data->size(); } //底层Quadlist的规模
16     int level() { return List::size(); } //层高 == #Quadlist，不一定要开放
17     bool put(K, V); //插入（注意与Map有别——Skiplist允许词条重复，故必然成功）
18     V* get(K k); //读取
19     bool remove(K k); //删除
20 };
```

代码9.2 Skiplist模板类

可见，借助多重继承（multiple inheritance）机制，由Dictionary和List共同派生而得的Skiplist模板类，同时具有这两种结构的特性；此外，这里还重写了在Dictionary抽象类（代码9.1）中，以虚函数形式定义的get()、put()和remove()等接口。

^② 由W. Pugh于1989年发明^[52]